

Cloud Provider Analytics

Proyecto Integrador - Minería de Datos II

Primera evaluación parcial Diseño y fundación de datos

ISTEA | 2C 2026

Apache Spark + PySpark | Google Colab

Patrón Lambda | Landing / Bronze / Silver / Gold

Repositorio: github.com/NickoGran/cloud-provider-analytics

1. Resumen ejecutivo

El proyecto representa al área de datos de un proveedor cloud. La solución propuesta integra fuentes batch y eventos de uso, preserva los datos originales, normaliza y conforma la información, y prepara productos analíticos para FinOps, Soporte y Producto/Usage.

Objetivo de la primera entrega: validar comprensión del caso, justificar la necesidad de Big Data y dejar una arquitectura viable y reproducible antes de la implementación profunda.

Usuarios y preguntas principales

Dominio	Necesidad analítica
FinOps	Costos, consumo, revenue, créditos, impuestos, anomalías y eficiencia.
Soporte	Tickets, severidad, SLA, CSAT y evolución por organización y fecha.
Producto / Usage	Uso de servicios, requests, GenAI, métricas operativas y carbono cuando exista.

Objetivos medibles de esta etapa

- Documentar el 100 % de las fuentes provistas.
- Definir grano, claves y principales riesgos de calidad.
- Distinguir claramente batch de streaming.
- Diseñar Landing, Bronze, Silver y Gold.
- Relacionar requisitos, 5V y componentes de arquitectura.
- Dejar evidencia reproducible en Colab y GitHub.

2. Evidencia y justificación Big Data - 5V

Dimensión	Evidencia / interpretación	Decisión
Volumen	43.200 eventos en la muestra; el escenario representa crecimiento continuo.	Spark + Parquet y arquitectura escalable.
Velocidad	120 archivos JSONL simulan micro-lotes; se requiere near real-time.	Structured Streaming para usage events.
Variedad	CSV + JSONL, múltiples dominios, servicios y métricas.	Landing + Bronze + Silver para preservar y conformar.
Veracidad	Nulos, tipos ambiguos, costos negativos y schema v1/v2.	Reglas de calidad y compatibilización en Silver.
Valor	FinOps, Soporte y Producto requieren salidas diferentes.	Marts Gold y serving en Cassandra/AstraDB.

Hallazgos del perfil inicial

Indicador	Resultado
Fuentes lógicas	8
Eventos de uso	43.200
Archivos JSONL	120
Schema v1 / v2	10.800 / 32.400
Duplicados en claves evaluadas	0
org_id sin maestro	0
resource_id sin maestro	0
value nulo	877 (2,03 %)
unit nulo	2.075 (4,80 %)
Costos negativos	216
value originalmente como texto	1.309

Nota: genai_tokens y carbon_kg presentan muchos nulos esperables por aplicabilidad y evolución de esquema; no deben interpretarse automáticamente como errores de calidad.

3. Arquitectura v1

Se adopta un patrón Lambda: batch para maestros/fuentes periódicas y streaming para usage events. Ambos caminos convergen en un único Data Lake y alimentan productos analíticos comunes.

Cloud Provider Analytics - Arquitectura v1

Primera evaluación - Patrón Lambda - PySpark + Data Lake + Serving

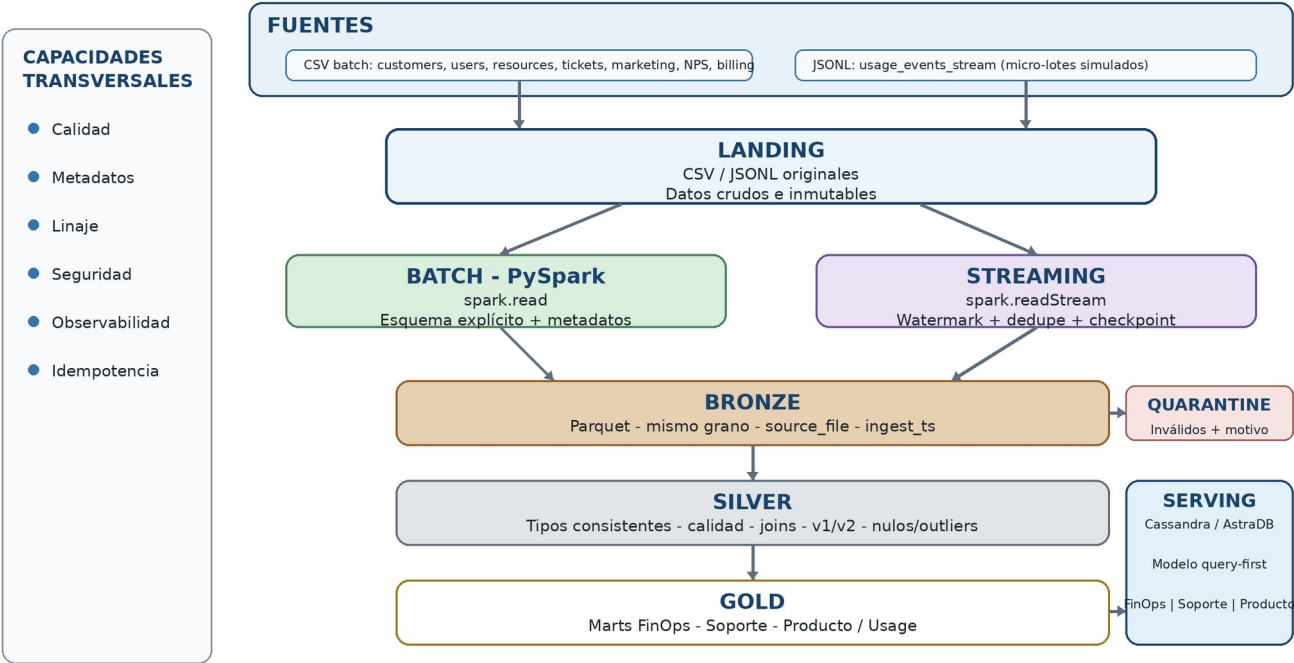


Figura 1. Arquitectura de alto nivel propuesta para la primera evaluación.

4. Diseño del Data Lake

Zona	Responsabilidad	Formato
Landing	Archivos originales e inmutables.	CSV / JSONL
Bronze	Mismo grano, tipificación explícita, source_file, ingest_ts y dedupe cuando aplique.	Parquet
Silver	Normalización, calidad, joins, nulos/outliers y compatibilidad v1/v2.	Parquet
Gold	Marts orientados a FinOps, Soporte y Producto/Usage.	Parquet
Quarantine	Registros que no superan reglas críticas, con motivo de rechazo.	Parquet

Particionamiento y naming

- Usage events: por fecha derivada de timestamp.
- Billing: por month.
- Gold diarios: por fecha; Gold mensuales: por mes.
- Maestros pequeños: evitar particiones de alta cardinalidad; usar fecha de ingesta solo si se justifica.
- Convención: minúsculas, snake_case, nombres descriptivos.

Reglas de promoción

Promoción	Criterio mínimo
Landing -> Bronze	Fuente conocida, lectura válida, esquema esperado, metadatos técnicos y mismo grano.
Bronze -> Silver	Tipos normalizados, calidad aplicada, claves válidas y schema compatibilizado.
Silver -> Gold	Calidad suficiente, dimensiones conformadas, grano definido y pregunta de negocio concreta.

5. Flujos batch y streaming

Aspecto	Batch	Streaming
Fuentes	Maestros, tickets, marketing, NPS, billing	usage_events_stream/*.jsonl
Lectura	spark.read	spark.readStream
Procesamiento	Ejecuciones periódicas	Micro-lotes incrementales
Esquema explícito	Sí	Sí
Deduplicación	Cuando corresponda	Por event_id
Watermark / late data	No aplica a las fuentes actuales	Sí
Checkpoint	No requerido como estado de streaming	Sí
Salida	Bronze Parquet -> Silver -> Gold	Bronze Parquet -> Silver -> Gold

Ejemplo batch: billing_monthly

billing_monthly.csv -> Landing -> PySpark -> Bronze Parquet/month -> Silver (tipos, credits, moneda, org_id) -> Gold
revenue_by_org_month -> Cassandra/AstraDB.

Ejemplo streaming: usage events

JSONL -> Landing -> Structured Streaming -> esquema explícito + watermark + dedupe + checkpoint -> Bronze -> Silver
(casting, v1/v2, calidad, joins) -> Gold por organización/fecha/servicio.

6. Procesamiento batch: MapReduce equivalente

Caso: costo diario por organización y servicio. Grano: org_id + usage_date + service.

Etapas conceptuales	Operación
Map	Emitir clave (org_id, usage_date, service) y valor cost_usd_increment.
Shuffle	Reunir registros con la misma clave.
Reduce	Sumar cost_usd_increment por grupo.
PySpark	groupBy(org_id, usage_date, service).agg(sum(cost_usd_increment)).

Trazabilidad requisito -> componente

Necesidad	Componente
Preservar originales	Landing
Estandarizar técnicamente	Bronze
Normalizar y enriquecer	Silver
Responder preguntas de negocio	Gold
Near real-time	Structured Streaming
Formato intermedio eficiente	Parquet
Consultas finales	Cassandra / AstraDB
Auditoría	source_file + ingest_ts + metadatos

7. Riesgos, mitigaciones y decisiones abiertas

Riesgo	Impacto	Mitigación
Schema v1/v2	Incompatibilidades	Conservar schema_version y compatibilizar en Silver.
Tipos ambiguos	Errores de cálculo	Casting controlado con esquema explícito.
Costos negativos / outliers	Métricas distorsionadas	Reglas de calidad y flag de anomalía.
Duplicados futuros	Doble contabilización	event_id + idempotencia.
Late data	Métricas temporales incompletas	Watermark y política de eventos tardíos.
Muchas particiones pequeñas	Menor eficiencia	Particionar solo con justificación; revisar repartition/coalesce.
Credenciales en repo	Riesgo de seguridad	Configuración externa y secretos fuera de GitHub.

Decisiones abiertas

- Watermark concreto de streaming.
- Método y umbral de anomalías.
- Cantidad óptima de particiones.
- Frecuencia exacta de algunas cargas batch.
- Retención productiva.
- Modelo físico query-first de Cassandra/AstraDB.

8. Esfuerzo, repositorio y próximos pasos

Etapas	Foco	Esfuerzo relativo
Evaluación 1	Diseño y fundación	25 %
Evaluación 2	Pipeline mínimo ejecutable	45 %
Final	Integración, dominios, pruebas y defensa	30 %

Repositorio

github.com/NickoGran/cloud-provider-analytics

- README.md: objetivo, arquitectura, ejecución y limitaciones.
- DECISIONS.md: decisiones técnicas y trade-offs.
- notebooks/: Colab de la evaluación.
- data/: instrucciones para obtener/cargar el dataset.
- evidence/: evidencias de ejecución.
- docs/: arquitectura_v1.png y este documento.

Checklist de la primera evaluación

Ítem	Estado
Problema, usuarios y objetivos	OK
5V	OK
Perfil de fuentes	OK
Arquitectura v1	OK
Patrón Lambda	OK
Matriz requisito-componente	OK
Data Lake	OK
Batch + streaming	OK
MapReduce equivalente	OK
Riesgos y mitigaciones	OK
Esfuerzo y recursos	OK
Repositorio y evidencia	OK

Próximo hito: incorporar feedback de la primera evaluación y evolucionar el diseño hacia un pipeline Landing -> Bronze -> Silver -> Gold -> Serving ejecutable.